

Міністерство освіти і науки України
Прикарпатський національний університет
імені Василя Стефаника
Факультет математики та інформатики
Кафедра інформаційних технологій

Програмування ігрових застосунків

ЗВІТ

про виконання завдання

**Тема: Розробка 2D-гри жанру Tower Defense
на платформі Unity з розгортанням у WebGL**

Виконав: Королюк В.А. ІПЗ-31

Викладач: Ровінський В. А.

1 ПОСТАНОВКА ЗАДАЧІ

Розробити двовимірну гру жанру Tower Defense на ігровому рушії Unity з цільовою платформою WebGL і онлайн-публікацією через GitHub Pages. Гравець виступає у ролі Захисника й будує оборонні вежі на сітці 12×8 проти автоматичних хвиль ворогів від AI-Атакуючого.

Перелік обов'язкових вимог:

- сітка 12×8 , фіксований шлях через точки-маршрути (waypoints);
- 4 типи веж (Лучник, Маг, Льодяник, Гармата) і 3 типи ворогів (Гоблін, Орк, Привид);
- автоматичний AI-противник, що формує склад хвилі в межах бюджету з ростом складності;
- 10 раундів, фінальні екрани перемоги/поразки, можливість рестарту;
- Object Pooling для ворогів і снарядів; стабільні 60 fps при 20 вежах + 50 ворогах;
- процедурна генерація спрайтів і звуків (без зовнішніх ассетів);
- WebGL-білд + публікація на GitHub Pages.

2 ОПИС ПРАВИЛ ТА ГЕЙМПЛЕЮ

Гравець-Захисник має 300 одиниць золота на старті. У фазі підготовки він купує і встановлює вежі на вільних клітинках. Після натискання кнопки «ПОЧАТИ ХВИЛЮ» починається фаза битви: AI-Атакуючий випускає ворогів з інтервалом 0,8–1,2 с. Кожен ворог, що дійде до бази, віднімає 1 НР. Стартове НР бази — 20. Перемога — пережити всі 10 раундів; поразка — НР=0. Бюджет AI зростає на +50 щораунд (стартує з 200).

2.1 Характеристики веж

Вежа	Ціна (g)	Швидкість	Шкода	Дальність	Тип атаки
Лучник	100	1,0 / с	20	3,0	Одна ціль

Маг	150	0,6 / с	30	2,0	АоЕ r=1,5
Льодяник	120	1,0 / с	10	3,0	Slow –50% / 2с
Гармата	200	0,3 / с	65	4,5	Одна ціль

2.2 Характеристики ворогів

Ворог	НР	Швидкість	Бюджет АІ	Винагорода	Особливість
Гоблін	40	3,0	10	5 g	—
Орк	150	1,5	25	12 g	висока стійкість
Привид	80	2,0	20	8 g	імунітет до Slow

2.3 Керування

Дія	Спосіб
Обрати вежу	ЛКМ по картці у нижньому барі
Поставити обрану вежу	ЛКМ по зеленій клітинці
Показати радіус дії вежі	ЛКМ по поставленій вежі
Скасувати вибір / радіус	ПКМ у полі
Розпочати хвилю	Кнопка «ПОЧАТИ ХВИЛЮ» у нижньому барі

3 СТРУКТУРА ПРЕФАБІВ І SCRIPTABLEOBJECT

3.1 Префаби

Hostile_<Type>	Turret_<Type>
└─ SpriteRenderer (sort=3)	└─ SpriteRenderer (sort=2)
└─ CircleCollider2D (trigger)	└─ BoxCollider2D (trigger)
└─ [Hostile]	└─ [Turret]
└─ [HostileLocomotion]	└─ [TurretFireControl]
└─ [HostileVitality]	
└─ HPCanvas (WorldSpace, sort=10)	
└─ Bar (Image, anchorMax.x — індикатор HP)	

HP-індикатор використовує не Slider, а пряму зміну anchorMax.x на Image — це надійніше для WorldSpace Canvas. Колір змінюється зелений → жовтий → червоний.

3.2 ScriptableObject-конфігурації

```
[CreateAssetMenu]
public sealed class TurretBlueprint : ScriptableObject {
    public string codename;
    public int credits;
    public float rateOfFire, reach, damage;
    public DamageProfile profile; // Direct | Splash | Chill
    public float splashRadius, chillFactor, chillSeconds;
    public GameObject turretPrefab, boltPrefab;
}
```

Створено 4 інстанси TurretBlueprint (Pulse, Plasma, Cryo, Rail з україномовними codename — Лучник, Маг, Льодяник, Гармата) і 3 інстанси HostileBlueprint (Scout → Гоблін, Tank → Орк, Phase → Привид). Усі ассети перестворюються при кожному запуску Vanguard → 1 — Build Match scene, щоб уникнути проблем з кешуванням SO на диску.

4 ЕКОНОМІКА, ХВИЛІ ТА ОПТИМІЗАЦІЯ

4.1 Алгоритм AI-Атакуючого

SwarmComposer формує чергу ворогів у межах бюджету. З 3-го раунду додає Орків, з 5-го — Привидів:

```
public Queue<HostileBlueprint> Compose(int round, int budget) {
    var queue = new Queue<HostileBlueprint>();
```

```

    int spent = 0;
    while (spent + scoutBP.budgetCost <= budget) {
        var pick = scoutBP;
        float r = Random.value;
        if (round >= 5 && r < 0.25f) pick = phaseBP;
        else if (round >= 3 && r < 0.55f) pick = tankBP;
        if (spent + pick.budgetCost > budget) pick = scoutBP;
        queue.Enqueue(pick);
        spent += pick.budgetCost;
    }
    return queue;
}

```

4.2 Object Pooling

Використано вбудований `UnityEngine.Pool.ObjectPool<T>`. При старті — pre-warm (15 Гоблінів, 8 Орків, 8 Привидів). Це усуває фриз при першому спавні і дозволяє витримати 50 одночасних ворогів + 20 веж без алокацій у головному циклі. Окремі пули — для ворогів (`HostilePool`) і для снарядів (`BoltPool`, по одному пулу на тип вежі).

Готова гра доступна онлайн за адресою:

<https://vladdddddw.github.io/towerDefense/>

Вихідний код:

<https://github.com/vladdddddw/towerDefense>

ВИСНОВКИ

У результаті виконання роботи розроблено повноцінну 2D-гру жанру Tower Defense на ігровому рушії Unity 6. Реалізовано всі поставлені завдання: машину станів з 5 фазами, 4 типи веж і 3 типи ворогів зі ScriptableObject-конфігураціями, AI-формування хвиль на 10 раундів, об'єктні пули ворогів і снарядів, процедурну генерацію спрайтів та аудіо. Гра успішно розгорнута у браузері через GitHub Pages та доступна за прямим посиланням. Проєкт демонструє практичне застосування шаблонів State Machine, Observer і Object Pool, а також особливостей розробки під WebGL-платформу.